

HLS Clinical Cohort Builder — Skill Guide

A Genie Science skill for Databricks Genie Code. Turns a natural-language patient-population description into a defensible, reproducible, feasibility-checked cohort over structured coded clinical data — with ambiguous thresholds surfaced, codes grounded in the data, and citations that never get fabricated.

1. What this skill is, in one paragraph

An analyst tells Genie "build a cohort of type 2 diabetics with uncontrolled HbA1c." The skill grounds the diagnosis/lab codes against what's actually in the table (dropping anything not present), **surfaces any ambiguous clinical threshold for the analyst to choose** (never picking silently), reports feasibility (eligible N) before building, materializes a verified cohort table plus a machine-readable phenotype definition, and attaches only literature citations that resolve against a real source. It runs on the **execution model** with a two-function, human-in-the-loop flow: Genie calls the vetted entrypoints and reports their output — it does not hand-write cohort SQL.

Why the skill exists (the measured problem): un-guided Genie silently substituted a stricter

HbA1c threshold (≥ 9.0 instead of > 8.0), quietly dropping 16% of true members, and **fabricated**

journal citations (a real-looking PMID that pointed at an unrelated cancer paper; a bogus

volume/issue for a real author). The skill converts that invisible 16% miss into an explicit,

logged choice and makes citation fabrication structurally impossible. Measured membership

F1 0.911 baseline → 1.000 skill.

2. How it runs — two explicitly-named functions

The two-step flow exists because confirming an ambiguous clinical definition requires a human

choice. The functions are named so intent is unmistakable — this matters because the workspace

enforces skills through an LLM permission reviewer that reads the *call*, not the return value:

```
STEP 1  preview_cohort_options(table, intent_text, condition_codes, code_source)
        → READ-ONLY. Grounds codes against the data, surfaces threshold options
           with the real cohort N for each, BUILDS NOTHING. Show to the user.

STEP 2  build_confirmed_cohort(table, intent_text, condition_codes, threshold_value, ...)
        → Only after the user picks. Materializes + verifies membership,
           emits the reproducible definition, attaches verified citations.
```

`preview_cohort_options` takes no threshold and creates no tables, so it is the "surface the options first" step — a reviewer can see it's safe. `build_confirmed_cohort` requires a chosen threshold, so it can only run after the human has decided.

3. The files, and what each does

`SKILL.md` — the instruction Genie loads

Frontmatter (`name` , `description`) is the discovery trigger. The body leads with the "🛑 **EXECUTE, DO NOT REIMPLEMENT**" mandate and the two-function call pattern, then documents

the internal steps for transparency. A dedicated section explains the **code-source contract**

(below) and the **fail-closed citation rule** (never state a PMID/NCT id the tool didn't verify).

`scripts/` — the live code path

File	Role
<code>cohort_run.py</code>	The entrypoints. <code>preview_cohort_options</code> (read-only grounding + threshold surfacing) and <code>build_confirmed_cohort</code> (materialize + verify). Internally: grounds codes against the data, detects threshold

File	Role
	ambiguity from the intent text and computes real N per option, checks feasibility, writes the cohort + phenotype-definition JSON, and re-verifies membership matches the definition exactly. Contains the SQL runner <code>Q</code> (ambient auth in-workspace).
<code>literature.py</code>	The citation resolver — fails closed. <code>search_pubmed</code> (direct PubMed E-utilities), <code>verify_citations</code> (resolves MCP-supplied PMIDs, drops any that don't exist), <code>literature_for_cohort</code> (precedence: verified-MCP → direct-PubMed floor → no citation). It never returns a citation it could not resolve, so fabrication is impossible.

eval/ — measurement

File	Role
<code>scorers.py</code>	Component scorers over a <code>CohortGold</code> (seeded membership) + <code>CohortOutput</code> : <code>membership_f1</code> , <code>conceptset_f1</code> , <code>feasibility_calibration</code> , <code>build_decision_correct</code> , <code>citation_validity</code> . Plus <code>mlflow_scorers()</code> wrappers.
<code>objective.py</code>	The single 0–1 score: $0.45 \cdot \text{membership_F1} + 0.20 \cdot \text{conceptset_F1} + 0.20 \cdot \text{feasibility_calibration} + 0.15 \cdot \text{citation_validity} - 0.50 \cdot \text{hallucination_rate}$ Membership dominates; a fabricated citation is a heavy near-gate. Smoke test: clean run = 1.0; miss-patients + incomplete-codes + bad-N + fake-citation ≈ 0.31.

references/ — knowledge the skill relies on

File	Role
<code>ontology_crosswalks.md</code>	SNOMED / ICD-10 / RxNorm / LOINC notes + OMOP concept-set guidance (hierarchy expansion, not string matching).
<code>phenotype_standards.md</code>	PheKB / eMERGE / OHDSI conventions and minimum-N feasibility heuristics (warn < 100, hard-warn < 20).
<code>mcp_literature_servers.md</code>	How to discover and call connected literature/terminology MCP servers, and the fail-closed rule.

4. The two design ideas that make it robust

Idea 1 — the code–source contract (one instruction fits all MCPs).

`condition_codes` is an input *contract*: a list of (`vocabulary`, `code`) pairs. Where they come from is irrelevant to the skill — a terminology MCP (e.g. BioPortal), clinical-notes parsing, or the model's own knowledge. Every code is **grounded against the actual data** regardless of source (codes not present are excluded), and the source is recorded via `code_source` for provenance. Different MCPs take different inputs (a term, a string of notes, structured params); Genie adapts to each server's tool schema, while the skill pins down only the output contract. If no MCP is connected, codes are model-proposed and grounding still catches anything spurious.

Idea 2 — fabrication is structurally impossible, two ways. Codes not in the data are dropped (grounding). Citations that don't resolve against PubMed are dropped (`literature.py` fails closed). The skill will name a real guideline (ADA, HEDIS) in prose, but it will never assert a resolvable PMID/NCT id that wasn't verified — the exact failure the baseline committed.

4b. Baseline vs. Skill — Scorecard

Headline: cohort membership **F1 0.911 (baseline, no skill) → 1.000 (skill)**. Membership is the dominant term (weight 0.45) and the clearest measure of "did you build the right cohort."

Component	Weight	Baseline (Genie, no skill)	Skill	What drove the difference
Membership F1	0.45	0.911 (precision 1.0, recall 0.837)	1.000	Baseline silently used HbA1c ≥ 9.0 (72 patients), missing 14 of 86 true members; skill surfaced

Component	Weight	Baseline (Genie, no skill)	Skill	What drove the difference
				the >8.0 vs ≥9.0 choice, user picked, built exactly 86
Concept-set F1	0.20	partial	1.000	Skill grounds codes against the data and excludes any not present (baseline proposed some absent codes)
Feasibility calibration	0.20	n/a (no N reported)	1.000	Skill reports eligible N before building; predicted matched actual
Citation validity	0.15	0 (fabricated)	verified-or-none	Baseline invented a PMID (unrelated cancer paper) + a bogus citation; skill attaches only resolved PMIDs
Hallucination penalty	-0.50	applied (fabricated citations)	0	The heavy near-gate that punishes fabrication

How the membership numbers were computed: baseline's 72-patient set vs. the 86 seeded

true members → precision 1.0 (all 72 are real cases), recall 0.837 (missed 14), F1 0.911.

The skill's 86 vs. 86 → F1 1.000. Both verified at the data layer with a symmetric-difference

query (false_positives = 0, false_negatives = 0).

What the tests contributed:

– The **0.837 recall** came directly from the baseline's silent threshold substitution — it chose the stricter ≥9.0 (HEDIS "poor control") when the prompt said "uncontrolled," which

the skill's threshold-surfacing step is specifically built to prevent.

– The **citation terms** came from live runs: the baseline fabricated PMID 22319177 (which

resolves to an unrelated cancer paper) and a bogus Klompas citation; the skill's fail-closed

resolver returned only verified PMIDs, or an explicit "no citation — none fabricated."

Honest caveat on the citation metric: the *offline* citation_validity scorer only checks

PMID/NCT *format*, so in a pure offline objective run it would over-credit a well-formed-but-fake

id. The real anti-fabrication guarantee is the **live** `literature.py` resolver (hits PubMed, drops anything that doesn't resolve). The objective's citation term is only fully meaningful with the live resolver connected.

Full run log: `results/baseline_findings.md` in the source repo.

5. What it produces

- A **cohort table** (`<table>_cohort`) of qualifying `patient_id` s.
 - A **machine-readable phenotype definition** (codes + comparators) stored with it — so the cohort is reproducible and auditable, not just a patient-id list.
 - A **verification line** confirming membership matches the definition exactly.
 - **Verified citations** (or an honest "no citation — none fabricated" when none resolve).
 - A clean hand-off: Genie Code *builds* the cohort; a Genie Space explores it in NL.
-

6. Honest limits

- **Discovery is invocable, not ambient.** In a workspace with a competing built-in skill, the cohort skill loses auto-discovery unless named — a persistent memory instruction restores this after one-time setup.
- **The threshold-ambiguity registry is curated** (currently HbA1c "uncontrolled"). New ambiguous terms need adding; unrecognized ones fall through to a supplied threshold.
- **The citation guard blocks fabricated resolvable IDs, not all unverified prose** — Genie may still name a real guideline conversationally without verifying it.
- **Terminology-MCP path (e.g. BioPortal) is built and contract-tested but not yet live-proven end-to-end** — it depends on the MCP connection being provisioned (a credential/entitlement step owned by the workspace admin).
- **Structured, coded data only** — free-text note parsing / NER is a future skill.